

Ficha de proyecto

Sistemas Embebidos

Sebastian Araneda - 27/04/2026

1. Plataforma a utilizar

Raspberry Pi Zero 2 W	
CPU	ARM Cortex-A53 quad-core 64-bit @ 1 GHz (SoC RP3A0)
RAM	512 MB LPDDR2 SDRAM
Factor de forma	65 mm × 30 mm
Consumo típico	~1.5 W en idle, ~2.5 W bajo carga. Alimentación 5 V / 2.5 A vía micro-USB
Precio referencial	~15 USD
Almacenamiento	microSD (sistema de archivos del SO y código)
Conectividad inalámbrica	Wi-Fi 802.11 b/g/n (2.4Ghz), Bluetooth 4.2 BLE
Buses de expansión	header GPIO de 40 pines (compatible con Raspberry Pi estándar)
Puertos físicos	1 × micro-USB OTG, 1 × mini-HDMI, 1 × conector CSI (cámara)
Protocolos de comunicación soportados	1-Wire, I2C, SPI, UART, GPIO digital/PWM, MQTT sobre Wi-Fi (capa de aplicación)

2. Usa sistema operativo: sí

2.1 Sistema operativo

Raspberry Pi OS Lite 64-bit (basado en Debian Bookworm). Se utiliza la versión Lite (sin entorno gráfico) para minimizar el uso de recursos.

2.2 Cargador de arranque

El proceso de arranque del Raspberry Pi no utiliza un bootloader convencional como GRUB o U-Boot. En su lugar, el proceso es gestionado por el firmware propietario del VideoCore IV (GPU integrada en el SoC). La secuencia es:

1. ROM de primera etapa en el SoC (inmutable, ejecutada por la GPU al encender)
2. bootcode.bin — segunda etapa, cargada desde la partición boot de la microSD
3. start.elf — firmware de la GPU, inicializa el hardware y carga el kernel Linux
4. kernel8.img — kernel Linux de 64 bits, entrega el control a la CPU ARM

La configuración del arranque se realiza mediante config.txt y cmdline.txt en la partición boot.

2.3 Software encargado de iniciar los procesos al arrancar systemd

Es el gestor de servicios del sistema. El script principal del sensor (sensor.py) se registra como un servicio de systemd (sensor.service), lo que garantiza que se inicia automáticamente después del arranque, que se reinicia ante fallos, y que gestiona las dependencias (por ejemplo, espera a que la red Wi-Fi esté disponible antes de iniciar).

2.4 Lenguajes de programación

Python 3 (script principal sensor.py): lectura de sensores, lógica de control, publicación MQTT y display OLED. Se eligió por el ecosistema de librerías disponibles para los periféricos usados (w1thermsensor, hx711, paho-mqtt, luma.oled) y porque el ciclo de 10 s hace que el overhead de Python sea irrelevante.

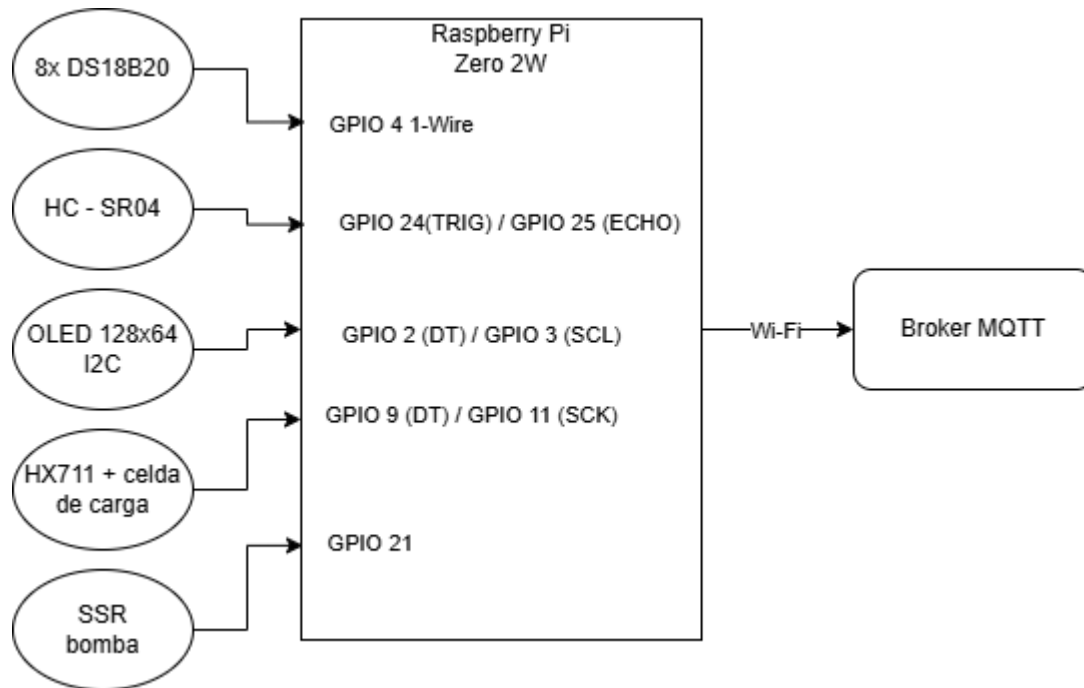
3. Descripción del desafío a abordar

En el contexto del monitoreo de tanques de almacenamiento de aceite de oliva, existe la necesidad de adquirir en tiempo real múltiples variables físicas del proceso: temperatura en distintas alturas del fluido, nivel de llenado y masa del contenido. Estos datos deben ser transmitidos de forma confiable a un sistema central para su procesamiento y visualización.

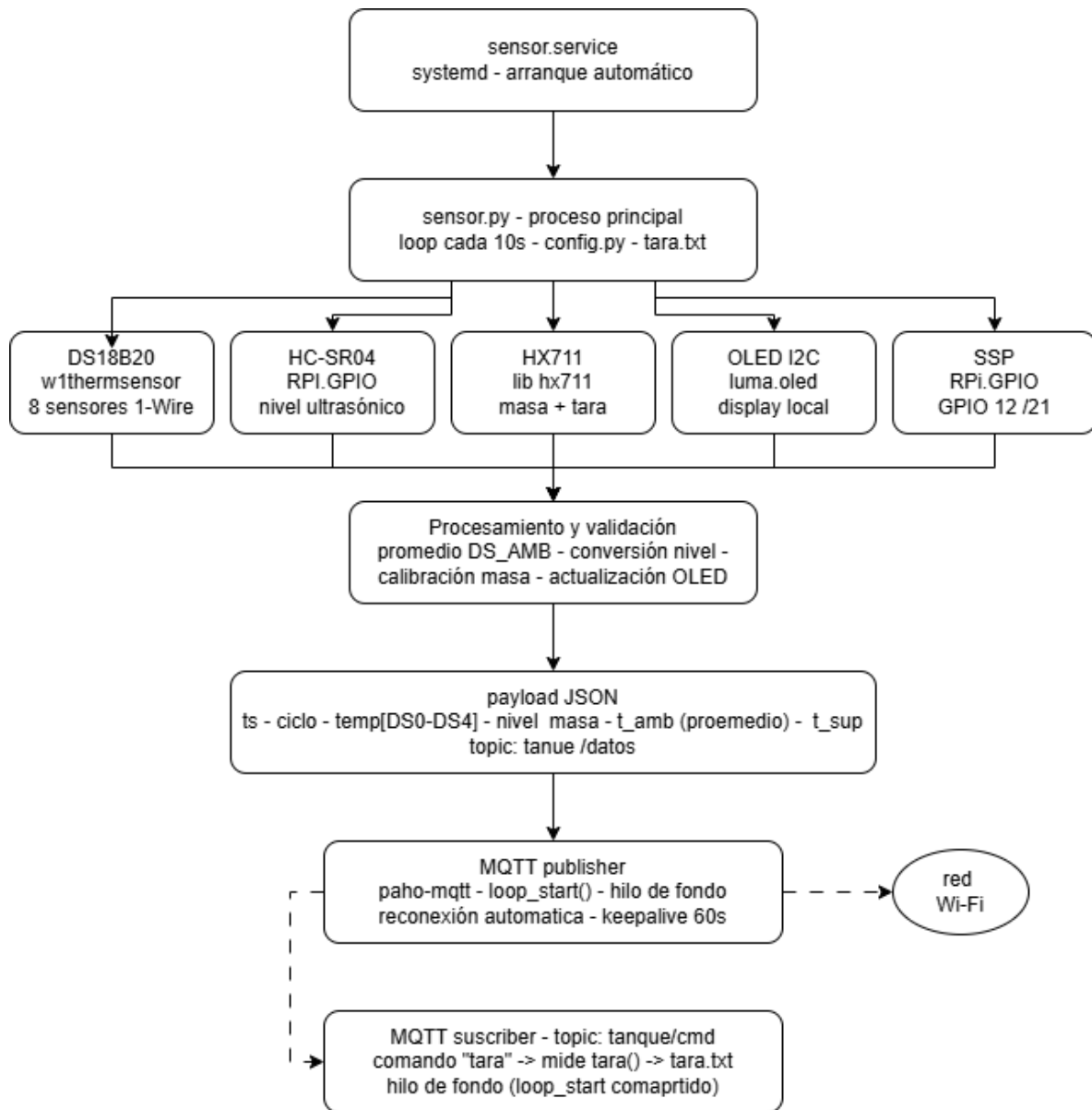
El desafío consiste en diseñar e implementar un nodo de adquisición embebido que sea capaz de leer simultáneamente sensores de distintos tipos y protocolos (1-Wire, ultrasonido, señal analógica mediante ADC externo, I²C), procesar y validar los datos, y transmitirlos vía MQTT a través de una red Wi-Fi, todo esto de forma continua, autónoma y con reinicio automático ante fallos. Adicionalmente, el nodo debe ser capaz de recibir comandos remotos para ejecutar acciones físicas (calibración de la balanza, control de una bomba de llenado).

4. Diagramas de bloques

4.1 Hardware



4.2 Software



5. Comunicación entre componentes de software

5.1 Comunicación interna (dentro del proceso sensor.py):

- El loop principal y el hilo del timer de la bomba comparten estado mediante variables globales (bomba_activa, _timer_llenado). El acceso concurrente es seguro en este caso porque las operaciones sobre estas variables son atómicas en Python (GIL) y el timer solo escribe al finalizar.
- La biblioteca paho-mqtt ejecuta el callback on_message en un hilo interno propio (loop_start), que se comunica con el hilo principal modificando variables globales y llamando funciones de control de GPIO.

5.2 Comunicación externa:

- Sensor -> Broker MQTT: el nodo publica un payload JSON cada 10 segundos al topic tanque/datos sobre Wi-Fi (TCP/IP, puerto 1883). El JSON contiene temperaturas, nivel, masa, estado de la bomba y temperatura ambiente/superior.
- Broker -> Sensor (comandos): el nodo está suscrito al topic tanque/cmd. Desde cualquier cliente MQTT en la red (Grafana, terminal, otro sistema) se pueden enviar comandos como bomba/llevar o tara.
- Sensor -> Broker (notificación al modelo): al finalizar el llenado automático, el nodo publica inicio/sup al topic modelo/cmd para notificar al gemelo digital que use la temperatura del tanque superior como condición inicial.
- Gestión de procesos: systemd es el mecanismo de comunicación entre el SO y el proceso Python. El archivo sensor.service define las dependencias, el comando de inicio, la política de reinicio y el archivo de entorno (variables de configuración).